



Productization Roadmap

Central Intelligence · Internal strategy · Virtual Assistants Philippines · 2026-07-17

TL;DR

~5-7 solo weeks to company #2 launch. Blocking phases: 0, 1, 2, 4 — phase 3 strongly recommended. Everything except phase 4 can be done before company #2's schema access arrives.

How Central Intelligence goes from a single-client deployment to a product any company can run — starting with company #2, without ever regressing the current client's instance. Approved 2026-07-15.

HARD CONSTRAINT

Greg's instance must never regress. Parity fixtures must hold on a staging clone before any production release, and releases are pinned to git tags.

1. Locked decisions

- **One deployment per company.** Each client gets their own droplet, app DB, Supabase auth project, and Vercel frontend. No multi-tenant rewrite, no `tenant_id` propagation. We provision instances (scripted); after that the client's admin self-serves all configuration in-app.
- **Company #2 is a different business entirely** — their Supabase won't match the current source schema. The sync layer becomes config-driven instead of hardcoded.
- **Flagship onboarding feature: AI-assisted mapping wizard** — introspect the client's database, AI proposes how their tables map into CI's concepts, a human approves, and the approved mapping drives backfill + hourly sync.
- **Schema access for company #2 is pending** — the discovery checklist ([docs/company2-discovery-checklist.html](#)) collects it; all prep phases proceed without it.

2. What blocks reuse today

#	BLOCKER	WHERE
1	Source-DB sync hardwired to the current client's schema (~28 tables, exact column names, stage value maps)	<code>backend/app/services/wgr_sync/</code>
2	Prompts hardcode a "coaching and consulting" vertical + literal person names; no per-instance business context	<code>backend/app/prompts/</code> (~18 files)
3	Metric SQL bound to current table shapes (incl. join quirks)	<code>backend/app/analytics/registry.py</code>
4	Three departments hardcoded as agent classes, routes, and nav	<code>agents/directors/</code> , <code>sidebar.tsx</code>
5	Currency "\$" and client-specific sync UI hardcoded in frontend	<code>insights/*</code> , <code>freshness-panel.tsx</code>

ALREADY REUSABLE

Client-DB connection config with read-only enforcement, UTC everywhere, centralized product branding, encrypted integration credentials, the generic upsert core, and the already-generic `DirectorAgent` base.

3. Key design choices

- **Canonical core, not dynamic schema.** `leads` , `calls` , `appointments` , `closed_sales` , `sales_reps` , `sales_activities` , `insights` , `email_campaigns` are generic business concepts any company maps onto. Client-flavored tables (coaching strikes, webinars, scorecards...) become optional **modules**, disabled per instance. Anything that fits nothing gets `entity_definitions` + `custom_records` (JSONB — syncable, RAG-embeddable, metric-queryable).
- **Generic sync engine as a new module.** The current client's pipeline is expressed as the engine's first mapping-config fixture for parity testing, but production stays on the legacy code until the engine proves parity. Company #2 launches on the engine from day one.
- **Metrics: hybrid.** The built-in metrics stay in code; a `metric_definitions` table adds per-instance metrics/overrides, with SQL validated against the existing contract and executed read-only with a statement timeout.
- **Prompts: templated.** A per-instance `instance_profile` table (vertical, description, terminology, benchmarks, currency, branding) is injected into every prompt as a business-context block. Literal person names are scrubbed.

4. Phases

BLOCKING FOR COMPANY #2'S LAUNCH

Phases **0, 1, 2, 4** (3 strongly recommended). Phases 5–7 are product depth. Everything except phase 4 can be done before their schema access arrives.

Phase 0 – Safety net + discovery kit DONE 2026-07-15 0.5–1 wk

- Tag `v1.0-greg-baseline` ; parity fixtures (prompt snapshots, 62 table counts, 14 metric values) in `backend/tests/parity/`
- `scripts/capture_parity_baseline.py` + staging parity runbook (`docs/staging-parity-runbook.html`)
- Discovery checklist ready to send to company #2

Phase 1 – Instance profile + prompt de-hardcoding BLOCKING 1–2 wk

- `instance_profile` singleton table + `routes/config.py` (profile, branding)
- `prompts/context.py` business-context block; ~18 prompt files templated; person names scrubbed
- Seed current client's profile with today's exact literals → rendered prompts byte-identical (snapshot test is the gate)
- Frontend: branding from API, currency helper replacing hardcoded "\$", `settings/business` page

Phase 2 – Provisioning + demo instance BUILT – IN REVIEW (PR #51) 1–2 wk

- Parameterized Caddyfile; `scripts/provision_instance.sh` + new-client runbook; `seed_instance.py`
- Rehearse end-to-end with a **demo instance** on fabricated data — permanent sales demo + testbed
- Verify all scheduled jobs no-op cleanly on an instance with no client-sync config

Phase 3 – Generic sync engine STRONGLY RECOMMENDED 2 wk

- `source_connections` (encrypted DSN) + versioned `sync_mappings` JSONB configs
- `services/sync_engine/` : spec validation, generalized read-only source client, transform registry, generic reader/mapper/upsert, parity differ
- Hourly `source_sync` task (skips when no connections); generic backfill script
- Current client expressed as a config fixture; shadow-run parity on staging; production stays on legacy sync

Phase 4 – Company #2 onboarding THE LAUNCH · GATED ON SCHEMA ACCESS 1–2 wk

- Run discovery; hand-author their mapping config (dry-runs the wizard's prompt design)
- `entity_definitions` / `custom_records` for anything non-canonical
- Provision their instance; seed profile/departments; `metric_definitions` table pulled forward — disable non-applicable built-ins, author 5–10 metrics for their business
- Backfill, hourly sync, integrations, branding; feature-verification test doc for a personal pass on real data

Phase 5 – AI mapping wizard

FLAGSHIP · NOT BLOCKING

2 wk

- `mapping_advisor` agent: introspection snapshot + canonical catalog + business profile → draft mapping with per-table confidence + rationale
- Onboarding routes: connect → introspect → propose → dry-run → activate
- `settings/data-source` wizard UI; freshness panel generalized per-connection
- Accuracy benchmark: score wizard proposals against both hand-approved configs

Phase 6 – Configurable departments + metrics UI

POLISH

1–2 wk

- `departments` table seeded with today's three verbatim; agent registry; `ConfigurableDirector` ; flag-flip migration then delete legacy director classes
- Navigation served from `GET /api/v1/config/navigation` ; sidebar fetches with static fallback
- Admin pages: `settings/departments` , `settings/metrics`

Phase 7 – Optional consolidation

LATER

- Cut the original instance over to the generic engine (parity already proven); retire legacy sync
- Wizard-driven re-mapping for schema drift; constrained metric-builder UI if raw-SQL authoring proves painful

5. The product pitch

1. **Private AI business brain** — dedicated single-tenant instance: your server, your database, never commingled with anyone else's.
2. **AI executive team** — an orchestrator plus configurable department directors and specialists that answer with numbers computed from your real data, never guessed.
3. **AI-assisted onboarding** — connect your database, AI proposes the data mapping, you approve, and backfill + hourly sync run automatically.
4. **KPI intelligence engine** — declarative metrics, daily snapshots, statistical improving/declining verdicts, threshold recommendations, weekly digest.
5. **Company knowledge base** — semantic search over Drive files, emails, call transcripts, and notes (RAG).
6. **Integrations hub** — Mailchimp, GoHighLevel, Google Workspace/Calendar/Drive, Meta/Google Ads, Instagram/Facebook; encrypted credentials, self-serve connection.
7. **Fully white-labeled** — your name, logo, colors, currency, terminology, and business context baked into every AI interaction.

6. Verification strategy

- **Per phase:** parity fixtures must hold on a staging clone before any production release (see the staging parity runbook); releases pinned to git tags.
 - **Phase 1:** prompt snapshot test — rendered-with-seed $\equiv$ old literals; chat smoke tests.
 - **Phase 3:** shadow-run parity report clean for every synced table; transform unit tests.
 - **Phase 4 (launch gate):** backfill counts match source; every metric validated on their data; directors answer real business questions correctly; `npm run build` before Vercel deploy; personal pass through the feature test doc on real data.
 - **Branch discipline:** fresh branch off latest main per phase; mirror synced after org merges before droplet redeploys.
-

© 2026 Virtual Assistants Philippines · AI Automation Powered